Listing 12.1 Wallet protocol, state, and sharding type key

```
object Wallet {

  val typeKey = EntityTypeKey[Command]("wallet")                              ❶

  sealed trait Command extends CborSerializable
  final case class ReserveFunds(
      amount: Int,
      replyTo: ActorRef[UpdatedResponse])
      extends Command                                                         ❷
  final case class AddFunds(amount: Int, replyTo: ActorRef[UpdatedResponse])  ❷
      extends Command                                                         ❷
  final case class CheckFunds(replyTo: ActorRef[Response]) extends Command    ❷

  sealed trait Event extends CborSerializable
  final case class FundsReserved(amount: Int) extends Event                   ❸
  final case class FundsAdded(amount: Int) extends Event                      ❸
  final case class FundsReservationDenied(amount: Int) extends Event          ❸

  sealed trait Response extends CborSerializable
  trait UpdatedResponse extends Response                                      ❹
  final case object Accepted extends UpdatedResponse                          ❹
  final case object Rejected extends UpdatedResponse                          ❹
  final case class CurrentBalance(amount: Int) extends Response               ❹

  final case class State(balance: Int) extends CborSerializable               ❺
  ...
}
```

❶ Key to be used as a sharding entity

❷ Commands

❸ Events

❹ Responses

❺ State

You need to make sure the state is durable so it can survive processes crashing or racks catching fire. To do that, you use Akka Persistence, adding the usual snapshotting and persistence failure.

Listing 12.2 Entity constructor